S.B. JAIN INSTITUTE OF TECHNOLOGY MANAGEMENT & RESEARCH, NAGPUR

Practical 10

Aim: Consider a disk queue with requests for I/O to blocks on cylinders 98, 183, 41, 122, 14, 124, 65, 67. The SSTF scheduling algorithm is used. The head is initially at cylinder number 53. The cylinders are numbered from 0 to 199. Write a Program to calculate the total head movement (in number of cylinders) incurred while servicing these requests.

Name: Pranav Paunikar

USN:CM24032

Semester / Year: **IV/II**

Academic Session: **2025-26**

Date of Performance:

Date of Submission:

- **Aim:** Consider a disk queue with requests for I/O to blocks on cylinders 98, 183, 41, 122, 14, 124, 65, 67. The SSTF scheduling algorithm is used. The head is initially at cylinder number 53. The cylinders are numbered from 0 to 199. Write a Program to calculate the total head movement (in number of cylinders) incurred while servicing these requests.
- **Objectives:**
  a) Implement the **Shortest Seek Time First (SSTF)** disk scheduling algorithm to optimize disk head movement.
  b) Calculate the total **head movement** required to service all I/O requests efficiently.
  c) Analyze the performance of SSTF by minimizing the seek time compared to other scheduling algorithms.
- **Requirements:**
  - ✓ **Hardware Requirements:**
    - Processor: Minimum 1 GHz
    - RAM: 512 MB or higher
    - Storage: 100 MB free space
  - ✓ **Software Requirements:**
    - Operating System: Linux/Unix-based
    - Shell: Bash 4.0 or higher
    - Text Editor: Nano, Vim, or any preferred editor

- **Theory:**

Disk scheduling algorithms are crucial in managing how data is read from and written to a computer's hard disk. These algorithms help determine the order in which disk read and write requests are processed, significantly impacting the speed and efficiency of data access. Common disk scheduling methods include First-Come, First-Served (FCFS), Shortest Seek Time First (SSTF), SCAN, C-SCAN, LOOK, and C-LOOK. By understanding and implementing these algorithms, we can optimize system performance and ensure faster data retrieval.

- Disk scheduling is a technique operating systems use to manage the order in which disk I/O (input/output) requests are processed.
- Disk scheduling is also known as I/O Scheduling.
- The main goals of disk scheduling are to optimize the performance of disk operations, reduce the time it takes to access data and improve overall system efficiency.

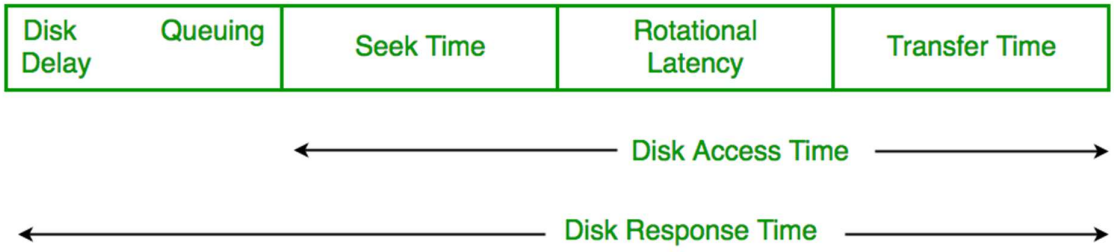
In this article, we will explore the different types of disk scheduling algorithms and their functions. By understanding and implementing these algorithms, we can optimize system performance and ensure faster data retrieval.

Importance of Disk Scheduling in Operating System

- Multiple I/O requests may arrive by different processes and only one I/O request can be served at a time by the disk controller. Thus other I/O requests need to wait in the waiting queue and need to be scheduled.
- Two or more requests may be far from each other so this can result in greater disk arm movement.
- Hard drives are one of the slowest parts of the computer system and thus need to be accessed in an efficient manner.

Key Terms Associated with Disk Scheduling

- **Seek Time:** Seek time is the time taken to locate the disk arm to a specified track where the data is to be read or written. So the disk scheduling algorithm that gives a minimum average seek time is better.
- **Rotational Latency:** Rotational Latency is the time taken by the desired sector of the disk to rotate into a position so that it can access the read/write heads. So the disk scheduling algorithm that gives minimum rotational latency is better.
- **Transfer Time:** Transfer time is the time to transfer the data. It depends on the rotating speed of the disk and the number of bytes to be transferred.
- **Disk Access Time:**
  Disk Access Time = Seek Time + Rotational Latency + Transfer Time
  *Total Seek Time = Total head Movement * Seek Time*



Disk Access Time and Disk Response Time

- **Disk Response Time:** Response Time is the average time spent by a request waiting to perform its I/O operation. The average *Response time* is the response time of all requests. *Variance Response Time* is the measure of how individual requests are serviced with respect to average response time. So the disk scheduling algorithm that gives minimum variance response time is better.

Goal of Disk Scheduling Algorithms

- Minimize Seek Time
- Maximize Throughput
- Minimize Latency
- Fairness

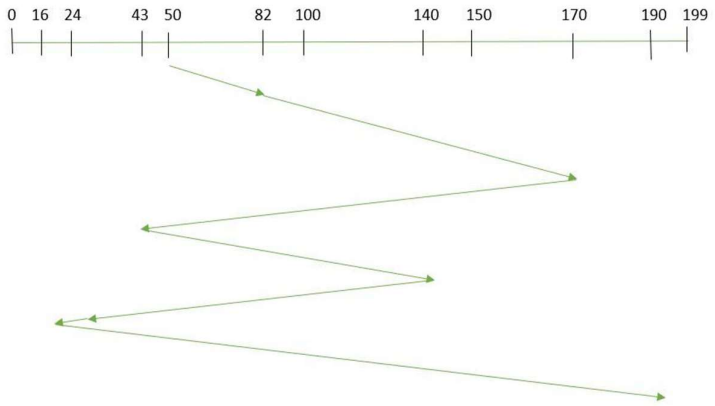
- Efficiency in Resource Utilization

  Disk Scheduling Algorithms

  There are several Disk Several Algorithms. We will discuss in detail each one of them.
- **FCFS (First Come First Serve)**
- **SSTF (Shortest Seek Time First)**
- **SCAN**
- **C-SCAN**
- **LOOK**
- **C-LOOK**
- **RSS (Random Scheduling)**
- **LIFO (Last-In First-Out)**
- **N-STEP SCAN**
- **F-SCAN**

1. FCFS (First Come First Serve)

FCFS is the simplest of all Disk Scheduling Algorithms. In FCFS, the requests are addressed in the order they arrive in the disk queue. Let us understand this with the help of an example.



First Come First Serve

Example:

Suppose the order of request is- (82,170,43,140,24,16,190)

And current position of Read/Write head is: 50

So, total overhead movement (total distance covered by the disk arm) = (82-50)+(170-82)+(170-43)+(140-43)+(140-24)+(24-16)+(190-16) =642

Advantages of FCFS

Here are some of the advantages of First Come First Serve.

- Every request gets a fair chance
- No indefinite postponement

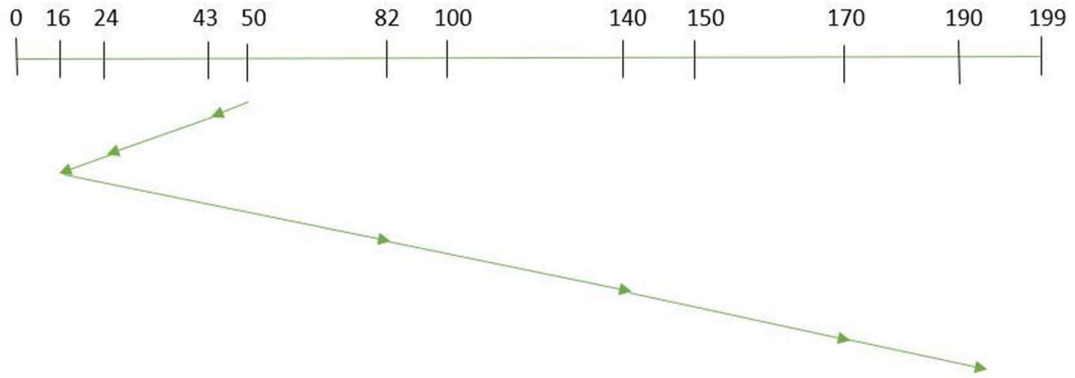
Disadvantages of FCFS

Here are some of the disadvantages of First Come First Serve.

- Does not try to optimize seek time
- May not provide the best possible service

2. SSTF (Shortest Seek Time First)

In SSTF (Shortest Seek Time First), requests having the shortest seek time are executed first. So, the seek time of every request is calculated in advance in the queue and then they are scheduled according to their calculated seek time. As a result, the request near the disk arm will get executed first. SSTF is certainly an improvement over FCFS as it decreases the average response time and increases the throughput of the system. Let us understand this with the help of an example.



Shortest Seek Time First

Example:

Suppose the order of request is- (82,170,43,140,24,16,190) And current position of Read/Write head is: 50

So,

total overhead movement (total distance covered by the disk arm) =

(50-43)+(43-24)+(24-16)+(82-16)+(140-82)+(170-140)+(190-170) =208

Advantages of Shortest Seek Time First

Here are some of the advantages of Shortest Seek Time First.

- The average Response Time decreases
- Throughput increases

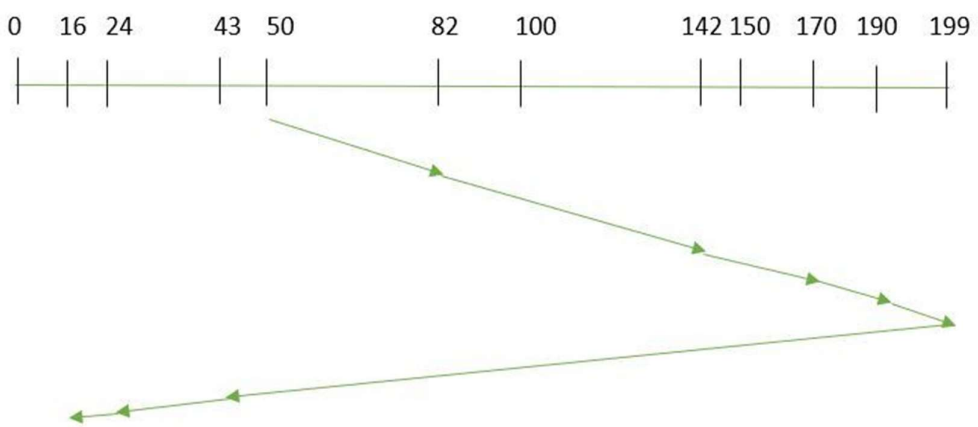
Disadvantages of Shortest Seek Time First

Here are some of the disadvantages of Shortest Seek Time First.

- Overhead to calculate seek time in advance
- Can cause Starvation for a request if it has a higher seek time as compared to incoming requests
- The high variance of response time as SSTF favors only some requests

3. SCAN

In the SCAN algorithm the disk arm moves in a particular direction and services the requests coming in its path and after reaching the end of the disk, it reverses its direction and again services the request arriving in its path. So, this algorithm works as an elevator and is hence also known as an **elevator algorithm.** As a result, the requests at the midrange are serviced more and those arriving behind the disk arm will have to wait.



SCAN Algorithm

Example:

Suppose the requests to be addressed are-82,170,43,140,24,16,190. And the Read/Write arm is at 50, and it is also given that the disk arm should move **"towards the larger value".**

Therefore, the total overhead movement (total distance covered by the disk arm) is calculated as

= (199-50) + (199-16) = 332

Advantages of SCAN Algorithm

Here are some of the advantages of the SCAN Algorithm.

- High throughput
- Low variance of response time
- Average response time

Disadvantages of SCAN Algorithm

Here are some of the disadvantages of the SCAN Algorithm.

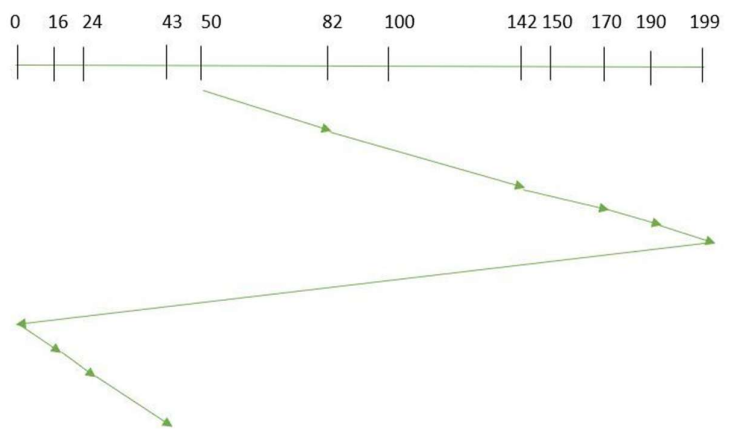
- Long waiting time for requests for locations just visited by disk arm

4. C-SCAN

In the SCAN algorithm, the disk arm again scans the path that has been scanned, after reversing its direction. So, it may be possible that too many requests are waiting at the other end or there may be zero or few requests pending at the scanned area.

These situations are avoided in the *CSCAN* algorithm in which the disk arm instead of reversing its direction goes to the other end of the disk and starts servicing the requests from there. So, the disk arm moves in a circular fashion and this algorithm is also similar to the SCAN algorithm hence it is known as C-SCAN (Circular SCAN).

Example:



Circular SCAN

Suppose the requests to be addressed are-82,170,43,140,24,16,190. And the Read/Write arm is at 50, and it is also given that the disk arm should move **"towards the larger value".**

So, the total overhead movement (total distance covered by the disk arm) is calculated as:

=(199-50) + (199-0) + (43-0) = 391

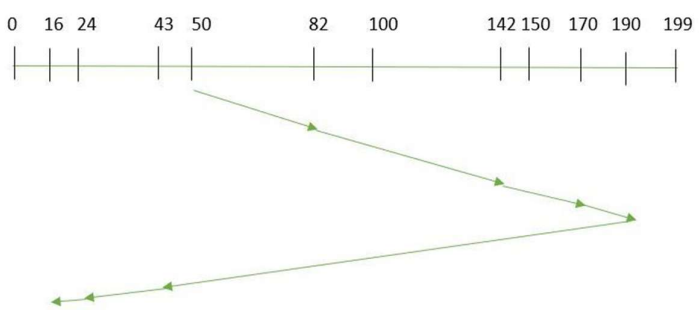
Advantages of C-SCAN Algorithm

Here are some of the advantages of C-SCAN.

- Provides more uniform wait time compared to SCAN.

5. LOOK

LOOK Algorithm is similar to the SCAN disk scheduling algorithm except for the difference that the disk arm in spite of going to the end of the disk goes only to the last request to be serviced in front of the head and then reverses its direction from there only. Thus it prevents the extra delay which occurred due to unnecessary traversal to the end of the disk.



LOOK Algorithm

Suppose the requests to be addressed are-82,170,43,140,24,16,190. And the Read/Write arm is at 50, and it is also given that the disk arm should move **"towards the larger value".**

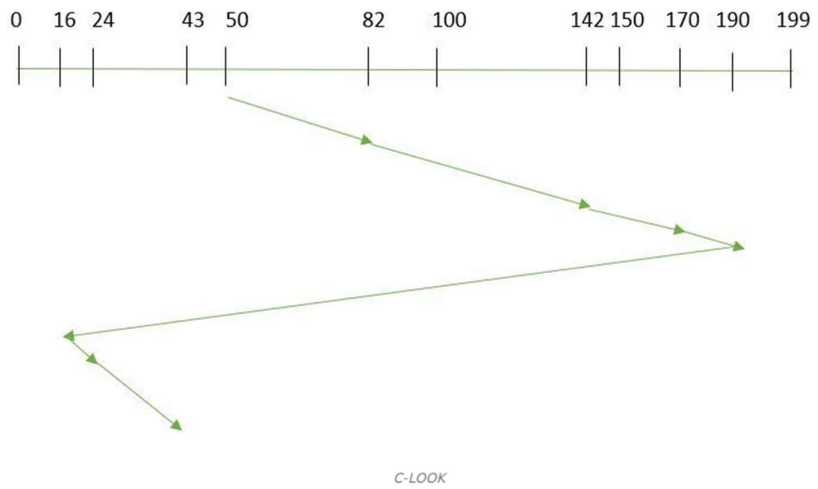
So, the total overhead movement (total distance covered by the disk arm) is calculated as:
= (190-50) + (190-16) = 314

6. C-LOOK

As LOOK is similar to the SCAN algorithm, in a similar way, C-LOOK is similar to the CSCAN disk scheduling algorithm. In CLOOK, the disk arm in spite of going to the end goes only to the last request to be serviced in front of the head and then from there goes to the other end's last request. Thus, it also prevents the extra delay which occurred due to unnecessary traversal to the end of the disk.

Example:

1. Suppose the requests to be addressed are-82,170,43,140,24,16,190. And the Read/Write arm is at 50, and it is also given that the disk arm should move **"towards the larger value"**



So, the total overhead movement (total distance covered by the disk arm) is calculated as
= (190-50) + (190-16) + (43-16) = 341

7. RSS (Random Scheduling)

It stands for Random Scheduling and just like its name it is natural. It is used in situations where scheduling involves random attributes such as random processing time, random due dates, random weights, and stochastic machine breakdowns this algorithm sits perfectly. Which is why it is usually used for analysis and simulation.

8. LIFO (Last-In First-Out)

In LIFO (Last In, First Out) algorithm, the newest jobs are serviced before the existing ones i.e. in order of requests that get serviced the job that is newest or last entered is serviced first, and then the rest in the same order.

Advantages of LIFO (Last-In First-Out)

Here are some of the advantages of the Last In First Out Algorithm.

- Maximizes locality and resource utilization
- Can seem a little unfair to other requests and if new requests keep coming in, it cause starvation to the old and existing ones.

9. N-STEP SCAN

It is also known as the N-STEP LOOK algorithm. In this, a buffer is created for N requests. All requests belonging to a buffer will be serviced in one go. Also once the buffer is full no new requests are kept in this buffer and are sent to another one. Now, when these N requests are serviced, the time comes for another top N request and this way all get requests to get a guaranteed service

Advantages of N-STEP SCAN

Here are some of the advantages of the N-Step Algorithm.

- It eliminates the starvation of requests completely

10. F-SCAN

This algorithm uses two sub-queues. During the scan, all requests in the first queue are serviced and the new incoming requests are added to the second queue. All new requests are kept on halt until the existing requests in the first queue are serviced.

Advantages of F-SCAN

Here are some of the advantages of the F-SCAN Algorithm.

- F-SCAN along with N-Step-SCAN prevents "arm stickiness" (phenomena in I/O scheduling where the scheduling algorithm continues to service requests at or near the current sector and thus prevents any seeking)

Each algorithm is unique in its own way. Overall Performance depends on the number and type of requests.

Note: Average Rotational latency is generally taken as 1/2(Rotational latency)

❖ **CODE:**

```
#include <stdio.h>

#include <stdlib.h>

#include <limits.h>

int main() {

    int requests[] = {98, 183, 41, 122, 14, 124, 65, 67};

    int n = sizeof(requests) / sizeof(requests[0]);

    int head = 53;

    int visited[8] = {0};

    int current_pos = head;
```

```
    int total_movement = 0;
    printf("--- SSTF DISK SCHEDULING TRACE ---\n");
    printf("Initial Position: %d\n\n", head);
    printf("%-15s %-15s %-15s\n", "From", "To", "Distance");
    printf("-------------------------------------------\n");
    for (int i = 0; i < n; i++) {
        int min_distance = INT_MAX;
        int index = -1;
        for (int j = 0; j < n; j++) {
            if (!visited[j]) {
                int distance = abs(requests[j] - current_pos);
                if (distance < min_distance) {
                    min_distance = distance;
                    index = j;
                }
            }
        }
        visited[index] = 1;
        printf("Cylinder %-7d -> Cylinder %-7d | Seek: %d\n",
                current_pos, requests[index], min_distance);
        total_movement += min_distance;
        current_pos = requests[index];  }
    printf("-------------------------------------------\n");
    printf("TOTAL HEAD MOVEMENT: %d cylinders\n", total_movement);
    return 0;
}
```

❖ **Output:**

```
--- SSTF DISK SCHEDULING TRACE ---
Initial Position: 53

From            To              Distance
-------------------------------------------
Cylinder 53      -> Cylinder 41      | Seek: 12
Cylinder 41      -> Cylinder 65      | Seek: 24
Cylinder 65      -> Cylinder 67      | Seek: 2
Cylinder 67      -> Cylinder 98      | Seek: 31
Cylinder 98      -> Cylinder 122     | Seek: 24
Cylinder 122     -> Cylinder 124     | Seek: 2
Cylinder 124     -> Cylinder 183     | Seek: 59
Cylinder 183     -> Cylinder 14      | Seek: 169
-------------------------------------------
TOTAL HEAD MOVEMENT: 323 cylinders
```

❖ **Conclusion**: In this practical, we conclude that the Shortest Seek Time First (SSTF) scheduling algorithm efficiently reduces total head movement by servicing the nearest cylinder request first. This approach minimizes seek time, improving disk performance compared to other scheduling methods.

❖ **Discussion Questions:**

1. What is the Shortest Seek Time First (SSTF) disk scheduling algorithm?
2. How does SSTF reduce the total head movement?
3. What is the drawback of SSTF scheduling?
4. How is total head movement calculated in SSTF?
5. How does SSTF compare to the FCFS scheduling algorithm?

❖ **References:**

https://www.geeksforgeeks.org/disk-scheduling-algorithms/

Date:____/____/2026

Signature
Course Coordinator
B.Tech CSE(AIML)
Sem: 4 / 2025-26